

# CryptoChain Analyzer Dashboard

## Final Report

Student: Claudia Lozano

This report summarizes the cryptographic metrics displayed in the dashboard, the AI component selected for the project, and the main conclusions obtained from the implementation.

### 1. Project Overview

This project consists of a real-time Streamlit dashboard connected to public Bitcoin blockchain APIs. Its purpose is to display live cryptographic metrics from the Bitcoin network and relate them to the theoretical concepts studied in class, especially Proof of Work, SHA-256 hashing, block headers, and difficulty adjustment. The application was designed as an interactive dashboard rather than a command-line script, in line with the assignment requirements.

The dashboard is divided into four main modules. M1 focuses on Proof of Work monitoring, M2 analyses the latest block header and verifies the Proof of Work locally, M3 studies Bitcoin difficulty adjustment history, and M4 adds an AI-based analytical layer to detect anomalous block inter-arrival times.

### 2. Cryptographic Metrics and Their Meaning

#### 2.1 M1 - Proof of Work Monitor

The first module displays live mining-related information from recent Bitcoin blocks. It includes the current difficulty, the number of leading zero bits in the block hash, the target threshold encoded by the bits field, and an estimated network hash rate. It also analyses the time between recent blocks and visualises their distribution.

Difficulty measures how hard it is to find a valid block compared with Bitcoin's original baseline difficulty. A higher difficulty means miners must perform, on average, more hashing work before finding a valid solution. The bits field is especially important because it stores the mining target in compact form. A valid block hash must be numerically smaller than this target, which directly connects the dashboard to the theory of Proof of Work.

This module also studies the time between consecutive blocks. In Bitcoin, the expected mean block interval is about 600 seconds. The distribution of recent inter-block times is useful because, in theory, block arrivals should approximately follow an exponential distribution. This makes the module valuable both as a real-time monitor and as a theoretical illustration of mining randomness.

#### 2.2 M2 - Block Header Analyzer

The second module focuses on the structure of the Bitcoin block header. It displays the six main header fields: version, previous block hash, Merkle root, timestamp, bits, and nonce. These fields make up the 80-byte header used in the Proof of Work process.

In this module, the header is reconstructed locally and hashed using  $\text{SHA256}(\text{SHA256}(\text{header}))$  with Python's hashlib library. The computed result is then compared against the block hash returned by the API. In addition, the bits field is decoded into the full target value, and the module checks whether the computed hash is below that target.

This is one of the most important parts of the project because it does not simply display API values: it independently verifies the Proof of Work with local code. The module also counts the number of leading zero bits in the resulting hash, which provides an intuitive way to understand why high-difficulty blocks are harder to mine.

## **2.3 M3 - Difficulty History**

The third module analyses Bitcoin difficulty over the latest completed adjustment periods. In Bitcoin, difficulty is adjusted every 2016 blocks in order to keep the average block production time close to 600 seconds. This module therefore studies the system at the level of full adjustment cycles rather than individual blocks.

For each completed adjustment period, the dashboard calculates the actual elapsed time between the beginning and the end of the period and compares it with the theoretical target time. This comparison is shown as a ratio between actual period time and target period time.

If the ratio is below 1, blocks were mined faster than expected during that period. If the ratio is above 1, they were mined more slowly than expected. This module shows how Bitcoin adapts dynamically to changes in total mining power while maintaining a stable long-term average block time.

## **3. AI Component**

### **3.1 Chosen Method**

The AI component implemented in this project is an anomaly detector for Bitcoin block inter-arrival times. Instead of predicting future difficulty or transaction fees, the system identifies unusually short or unusually long block intervals compared with the rest of the analysed sample.

### **3.2 Why This Method Was Chosen**

This method was chosen because it is strongly connected to the theory of Bitcoin mining. Since block arrivals are expected to behave approximately like an exponential process with an average around 600 seconds, strong deviations can be interpreted as statistically unusual events. This makes anomaly detection both theoretically meaningful and practical to visualise in a dashboard.

Another reason for choosing this method is interpretability. In an academic setting, it is important to understand and explain every part of the system. A transparent anomaly detector is easier to justify than a more complex model with less explainable behaviour.

### **3.3 Data Used**

The anomaly detector uses recent Bitcoin blocks obtained from a public API. For each pair of consecutive blocks, the inter-arrival time in seconds is computed. These values form the dataset used by the model. For each analysed interval, the dashboard stores the block height, timestamp, inter-arrival time, deviation from the expected 600-second mean, and the corresponding z-score.

### **3.4 Method and Evaluation**

The implemented method is based on z-scores. First, the mean and standard deviation of the observed inter-arrival times are calculated. Then each interval is transformed into a z-score. Intervals with an absolute z-score greater than 2 are flagged as anomalous.

This is a simple statistical baseline rather than a large machine learning model, but it still adds analytical value because it automatically highlights unusual timing behaviour from real blockchain data. The dashboard reports how many intervals were analysed, how many were flagged as anomalous, and the average observed block time in the selected sample.

The main limitation of this method is that it is purely statistical. A flagged interval is unusual with respect to the sample distribution, but that does not necessarily mean that it reflects a real

protocol-level problem. Still, the method is useful as a lightweight and explainable AI component.

## 4. Conclusions

This project demonstrates how theoretical cryptographic concepts can be connected to real blockchain data in a practical dashboard. The implemented modules show the meaning of difficulty, target thresholds, block headers, double SHA-256 hashing, and Bitcoin's difficulty adjustment mechanism.

The strongest technical part of the project is the local verification of Proof of Work in M2, because it proves that the application is not only displaying values returned by an API but also independently validating the cryptographic process. M3 complements this by showing how Bitcoin regulates mining difficulty over time, while M4 adds an analytical layer that detects statistically unusual block timing behaviour.

Overall, the project combines live data collection, cryptographic verification, historical analysis, and AI-based anomaly detection in a single interactive application.

## References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
2. Blockstream API Documentation.
3. Blockchain.com API Documentation.